CSC 3320: System-Level Programming

Chapter 3, 4

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

Why float, double, long double?

They are for precision of computation and bigger range:

```
#include <stdio.h>

int main() {
    float f = 1.0f / 3.0f;
    double d = 1.0 / 3.0;
    long double ld = 1.0L / 3.0L;

    printf("float:       %.20f\n", f);
    printf("double:      %.20lf\n", d);
    printf("long double: %.20Lf\n", ld);

    return 0;
}
```

```
Output:
float:       0.33333334326744079590
double:      0.33333333333333331483
long double: 0.33333333333333333334
```

Chapter 3 – (contd.)

Confusing `printf` and `scanf` statement

```
printf("%d %d\n”, &i, &j); /*** WRONG ***/
```

What is wrong in this printf statement?

Note: `scanf` normally skips white-space characters when looking for data items, better to use only conversion specification (no other characters) in a format string to avoid errors.

```
scanf("%d, %d", &i, &j);
```

What will happen in this case?

Exercise

```
#include <stdio.h>

int main(void)
{
    int i; float x;
    i = 40;
    x = 839.21f;
    printf(" %d|%5d %-5d|%5.3d|\n", i, i, i, i);
    printf("%10.3f %10.3e|%-10g|\n", x, x, x);
    return 0;

}
```

```
Output:
 40|   40 40   |  040|
   839.210  8.392e+02|839.21    |
```

Exercise

In response to the input statement

```
scanf("%4d %*d%d", &year, &code, &count);
```

Skip other digits

the following data is keyed in 123456 76

What values does the computer assign to the variables year, code and count?

Program: Adding Fractions

The `addfrac.c` program prompts the user to enter two fractions and then displays their sum.

Sample program output:

```
Enter first  fraction:  5/6
Enter second fraction:  3/4
The  sum is 38/24
```

addfrac.c

```
/* Adds two fractions */
#include<stdio.h>
int main(void)
{
  int num1, denom1, num2, denom2;
  int result_num, result_denom;

  printf("Enter first fraction: ");
  scanf("%d/%d", &num1, &denom1);

  printf("Enter second fraction: ");
  scanf("%d/%d", &num2, &denom2);

  result_num = num1 * denom2 + num2 *denom1;
  result_denom = denom1 * denom2;
  printf("The sum is %d/%d\n",result_num, result_denom) ;

  return 0;
}
```

Chapter 4: Expressions

Expressions

Arithmetic operator

+, -, *, /, %, ++, --.....

Relational operator

<, >, <=, >=, !=

- Note: In Java, they return values FALSE and TRUE
- In C, they return values 0 and 1
  - **0** means *false*, a **non-zero** values including 1 means *true*
  - E.g. any value below can represent *true*
  - *1.1, -0.5, 1, 10, 3.1415926*

Operator Precedence

Operator Precedence

Highest:	+		-	(unary)
	*	/	%	
Lowest:	+		-	(binary)

Examples of unary operations

+3 -5

Binary operations

2 + 3 4 - 5

Precedence and Associativity

`i + j * k`	is equivalent to	`i + (j * k)`
`-i * -j`	is equivalent to	`(-i) * (-j)`
`+i + j / k`	is equivalent to	`(+i) + (j / k)`

An operator is said to be left associative if it groups from left to right. The binary arithmetic operators (*, /, %, +, and -) are all left associative.

`i - j - k`	is equivalent to	`(i - j) - k`
`i * j / k`	is equivalent to	`(i * j) / k`

An operator is right associative if it groups from right to left. The unary arithmetic operators (+ and -) are both right associative.

`- + i` is equivalent to `- (+i) // Try: printf("%d", - + - 5);`

Assignment Operator

Since assignment is an operator, several assignments can be chained together:

```
i = j = k = 0;
```

The = operator is right associative, so this assignment is equivalent to

```
i = (j = (k = 0));
```

The effect is to assign 0 first to k, then to j , and finally to i.

```
12 = i; /*** WRONG ***/
i+j=0; /*** WRONG ***/
-i=j; /*** WRONG ***/
```

L-value (left value required for assignment operator)

Compound Assignment

The compound assignment operators have the same properties as the = operator. They're right associative, so the statement

```
i += j += k;  means  i += ( j += k )
```

Increment-Decrement Operators

```
i = 1;
printf("i is %d\n", ++i);      /* prints "i is 2" */
printf("i is %d\n", i);        /* prints "i is 2" */
```

Evaluating the expression `i++` (a "post-increment") produces the result i, but causes i to be incremented afterwards:

```
printf("i is %d\n", i++);      /* prints ”i is 1" */
printf("i is %d\n", i);        /* prints ”i is 2” */
```

```
i = 1;
printf("i is %d\n", --i);  /* prints ”i is 0” */
printf("i is %d\n",i);     /*prints ”i is 0”*/
i = 1;
printf(”i is %d\n",i--);   /*prints ”i is 1”*/
printf(”i is %d\n", i);    /* prints "i is 0" */
```